Joint Model Selection and Adaptation for Few-Shot Digital Twin Instantiation under Deployment Limits

ARTICLE INFO

Keywords:
digital twin
few-shot learning
model instantiation
architecture selection
model deployment limits
heterogeneous computing centers

ABSTRACT

Digital twin (DT) platforms for computing networks can reuse pretrained models from existing centers to reduce the computation required for DT instantiation. However, a new or reconfigured center may exhibit substantial differences in resource configurations and workload characteristics, making a fixed architecture suboptimal for the target environment. At the target center, only a small support set is available for parameter adaptation and a small validation set for model selection. Accurate DT instantiation therefore requires selecting a suitable architecture and adapting its parameters from such limited target data while ensuring that the selected model satisfies deployment limits. Existing few-shot methods typically adapt parameters within predefined architectures and rarely account for deployment limits, limiting their effectiveness in heterogeneous computing environments. To address this challenge, we propose Model Selection and Adaptation for Digital Twin Instantiation (MSA-DTI), a few-shot DT instantiation framework that jointly selects a target-specific architecture and adapts its parameters under deployment limits. MSA-DTI first constructs a candidate model bank by pretraining diverse architectures on historical workload data, filters out infeasible candidates according to the deployment limits, and adapts the remaining candidates using limited target samples. To make model selection more conservative given this small validation set, it compares the best-performing adapted alternative with a fixed adapted reference model and uses a preset relative reduction in validation loss as the replacement criterion. Experimental results show that MSA-DTI achieves the lowest mean squared error (MSE) among the compared methods, reducing MSE by 14.60% on average across paired cases relative to the pretraining-and-fine-tuning baseline while satisfying the deployment limits across all main evaluation cases.

1. Introduction

A shared digital twin (DT) platform maintains center-specific predictive models for monitoring, prediction, analysis, and decision making [1–3]. These models transform synchronized monitoring histories into workload forecasts that can support resource-management decisions. The present work addresses this predictive-model component; sensing, state synchronization, simulation, and actuation remain services of the surrounding DT platform. To reduce the computation required for model instantiation, the platform can reuse models pretrained on existing centers. However, a new or reconfigured center may differ substantially from existing centers in resource configurations, monitored-variable availability, workload patterns, and data distributions, making a fixed pretrained architecture suboptimal for the target environment. Concurrent model maintenance also creates computation and storage pressure, so the platform sets two per-model deployment limits: estimated operation count per inference and parameter count. The operation count describes model-side inference computation, while the parameter count describes model size. In this setting, the target center has only limited historical observations, available as a small support set for parameter adaptation and a small validation set for model selection. The limited support set defines the few-shot target-adaptation setting considered in this work. Selecting a suitable architecture and adapting its parameters from such limited target data are challenging, while the resulting DT model must satisfy both deployment limits. Figure 1 shows this setting.

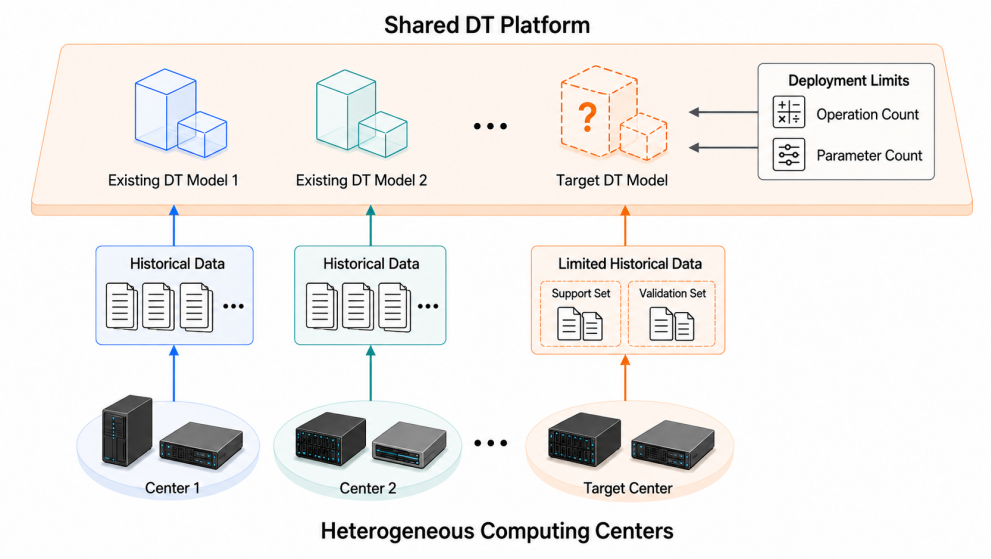


Figure 1: System setting for few-shot digital twin model instantiation with limited target data and deployment limits.

Existing parameter-adaptation-based few-shot and meta-learning methods adapt model parameters from limited data [9, 10], and few-shot adaptation has also been applied to DT modeling [11]. These methods usually keep the model architecture fixed and do not explicitly incorporate per-model deployment limits into target-side adaptation, which limits their applicability to heterogeneous target centers. Deployment-aware architecture selection considers model performance and deployment cost [12, 13, 26], while meta-learning has also been used to adapt architectures and fine-tuning strategies to new few-shot tasks [30–32]. Such methods learn or search an architecture or adaptation structure for the target task, and H-Meta-NAS further considers hardware-constrained architecture adaptation [30]. Unlike these search-based settings, the target-side task here starts from the pretrained models already available at the DT platform. It therefore concerns selection and adaptation of reusable models under deployment limits rather than target-side architecture or adaptation-structure search.

To address this problem, we propose Model Selection and Adaptation for Digital Twin Instantiation (MSA-DTI). MSA-DTI first builds a candidate bank, defined as a set of models pretrained on historical data from existing centers, and removes candidates that violate the target deployment limits. It then adapts all feasible candidates using the same support set and update budget. Because the target validation set is small, MSA-DTI compares the best-performing adapted alternative with a fixed adapted reference model and uses a preset relative reduction in validation loss as the replacement criterion. This design enables target-specific architecture selection without target-side architecture search and prevents replacement based only on marginal validation-loss reductions. The selected architecture and adapted parameters form the predictive-model instance used by the target center's DT.

The target optimization object is a finite set of architecture–initialization pairs after common target adaptation. This differs from ordinary model-bank selection, which scores frozen bank entries, and from PT+FT, which adapts only one fixed reference architecture. It also differs from NAS and meta-NAS, which search or adapt an architecture in the target stage. The relative margin is a separate final-selection rule: it does not construct the bank, change the target updates, or create the gain from cross-architecture adaptation.

This work makes the following contributions.

- **Problem formulation.** We formulate few-shot DT model instantiation for a target center as architecture selection and parameter adaptation from limited target data, subject to per-model operation-count and parameter-count limits.
- **MSA-DTI algorithm.** We develop MSA-DTI to filter infeasible candidates, adapt feasible candidates under a common target budget, and exhaustively evaluate the finite feasible set for target model selection.
- **Reference-based selection.** We introduce a calibrated relative-margin rule that replaces the fixed adapted reference only when the best alternative achieves sufficient validation-loss reduction.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 defines the system model and problem. Section 4 presents MSA-DTI. Section 5 reports the experiments, and Section 6 concludes the paper.

2. Related Work

2.1. Digital Twin Modeling and Platform Deployment

Digital twin research uses virtual models for monitoring, prediction, simulation, and control [1–4]. In communication and computing systems, digital twins also represent system states and services and support model updates [5–8]. Platform studies further consider service orchestration and computation offloading under resource constraints [27, 28]. However, these studies usually assume a predefined model structure or focus on platform-level resource decisions rather than target-specific architecture selection and parameter adaptation from limited target data under deployment limits.

2.2. Few-Shot Learning and Meta-Learning

Few-shot learning and meta-learning reuse prior knowledge to adapt models from limited data [9, 10, 24]. Recent methods improve cross-task and cross-domain adaptation through context modeling and transferable representations [20–23], and limited-data learning has also been applied to DT construction and workload prediction [11, 25]. Most parameter-adaptation methods keep the architecture fixed and mainly adapt model parameters, without explicitly accounting for per-model deployment limits, which limits their applicability to heterogeneous target centers.

2.3. Deployment-Aware Architecture Selection

Deployment-aware neural architecture search evaluates model performance together with platform or hardware cost [12, 13, 26]. Few-shot, zero-shot, and training-free NAS methods reduce architecture evaluation or search cost [14–16]. In Ref. [14], few-shot refers to the use of several sub-supernets for architecture evaluation rather than adaptation from a small target support set.

Meta-learning has also been combined with architecture search. T-NAS learns a transferable meta-architecture that is adapted to new tasks [31], while H-Meta-NAS extends task-adaptive architecture search to hardware-constrained deployments [30]. Neural fine-tuning search instead starts from a pretrained backbone and searches the adaptation structure, including which components to fine-tune or augment [32]. These studies show that architecture decisions and few-shot adaptation can be coupled.

MSA-DTI addresses a different target-side protocol. Unlike methods that learn or search a target architecture or adaptation structure, it operates over a finite bank of separately source-trained full-model candidates. Deployment feasibility is determined before target adaptation, whereas candidate preference is evaluated only after the feasible candidates have been adapted with the limited target support set. The resulting problem is therefore target-side instantiation over reusable frozen models under deployment limits rather than target-side architecture or adaptation-structure search.

3. System Model and Problem Formulation

3.1. System Setting and Target Model

We consider a digital twin platform that maintains center-specific DT models for heterogeneous computing centers. Let $\mathcal{C}_{\mathrm{src}}$ denote the existing centers, and let $\mathcal{D}_{\mathrm{src}}$ denote their combined historical workload data.

For each instantiation case, let c denote a new or reconfigured target center. Let $\mathcal{D}_c^{\mathrm{sup}}$ and $\mathcal{D}_c^{\mathrm{val}}$ denote its support and validation sets introduced in Section 1. All centers are mapped to a common platform-wide metric schema. A center may observe only a subset of the metric positions, but the input dimension remains fixed after unavailable positions are masked.

Let $\mathcal{A}$ denote a predefined architecture space. For $a \in \mathcal{A}$, let $f_{a,\theta}$ denote a model with parameter vector θ. The predictive-model component instantiated for the DT of center c is

$$\mathrm{DT}_c = \left(a_c, \theta_c\right), \tag{1}$$

where a_c is the selected architecture and θ_c is its adapted parameter vector. We retain the notation DT_c as shorthand for this component; it does not represent the complete DT system. Model instantiation must therefore determine both terms.

3.2. Candidate Models and Deployment Limits

A candidate model is written as $q = (a_q, \theta_q^0)$, where $a_q \in \mathcal{A}$ is its architecture and θ_q^0 is its parameter vector before target adaptation. Let $\mathcal{Q}$ denote a finite set of candidates, called the candidate bank.

For target center c, let B_c^F and B_c^P denote the operation-count and parameter-count limits, respectively. Let $F_c(a_q)$ be the estimated operation count of architecture a_q for one inference, and let $P_c(a_q)$ be its parameter count. The same counting rule is used for all candidates; the complete rule is given in Supplementary Section S1.

The candidates satisfying both limits form

$$\mathcal{Q}_c^{\mathrm{fea}} = \left\{ q \in \mathcal{Q} \;\middle|\; F_c(a_q) \le B_c^F,\ P_c(a_q) \le B_c^P \right\}. \tag{2}$$

Here, $\mathcal{Q}_c^{\mathrm{fea}}$ is the feasible candidate set. Both deployment limits are hard constraints rather than penalty terms in the task loss. The two counts are deterministic model-level deployment indicators rather than direct guarantees of hardware latency or memory use. Their relation to measured latency and model size is evaluated in Supplementary Section S5.

MSA-DTI uses a fixed reference candidate q^{ref} as the fallback against which alternative candidates are evaluated. The reference is fixed offline and is not re-selected using target data. We define a target limit setting as supported when $q^{\mathrm{ref}} \in \mathcal{Q}_c^{\mathrm{fea}}$. If the reference candidate violates an assigned limit, MSA-DTI reports UnsupportedLimit rather than changing the reference after observing target validation data. This preserves the fixed reference used by the calibrated selection rule.

3.3. Few-Shot Model Instantiation Problem

Let $\mathcal{L}(\mathcal{D}; a, \theta)$ denote the mean task loss of model $f_{a,\theta}$ on dataset $\mathcal{D}$. Let T denote the number of target updates. For a feasible candidate q, let $\theta_{c,q}^{(T)}$ denote its parameters after T updates on $\mathcal{D}_c^{\mathrm{sup}}$.

Its validation loss after adaptation is

$$J_c(q) = \mathcal{L}\left(\mathcal{D}_c^{\mathrm{val}}; a_q, \theta_{c,q}^{(T)}\right). \tag{3}$$

For a supported target limit setting, the few-shot model instantiation problem is to determine the target DT instance from $\mathcal{Q}_c^{\mathrm{fea}}$ using only $\mathcal{D}_c^{\mathrm{sup}}$ and $\mathcal{D}_c^{\mathrm{val}}$. This requires selecting a suitable architecture and adapting its parameters from limited target data while the returned model remains within both deployment limits. The selected candidate determines a_c and θ_c in Eq. (1).

We solve this problem using MSA-DTI, whose target-side procedure consists of fixed-budget parameter adaptation, exhaustive evaluation over the finite feasible candidate set, and reference-based final selection.

4. The Proposed MSA-DTI Method

4.1. Method Overview

Model Selection and Adaptation for Digital Twin Instantiation (MSA-DTI) has three stages. Offline preparation screens the architecture space on architecture-screening cases, constructs the candidate bank, fixes the reference candidate, and calibrates the relative margin on a disjoint margin-calibration pool. Deployment filtering removes candidates that do not satisfy the target limits. Target adaptation and selection then adapt the remaining candidates and select the final model.

Figure 2 shows the overall three-stage workflow of MSA-DTI.

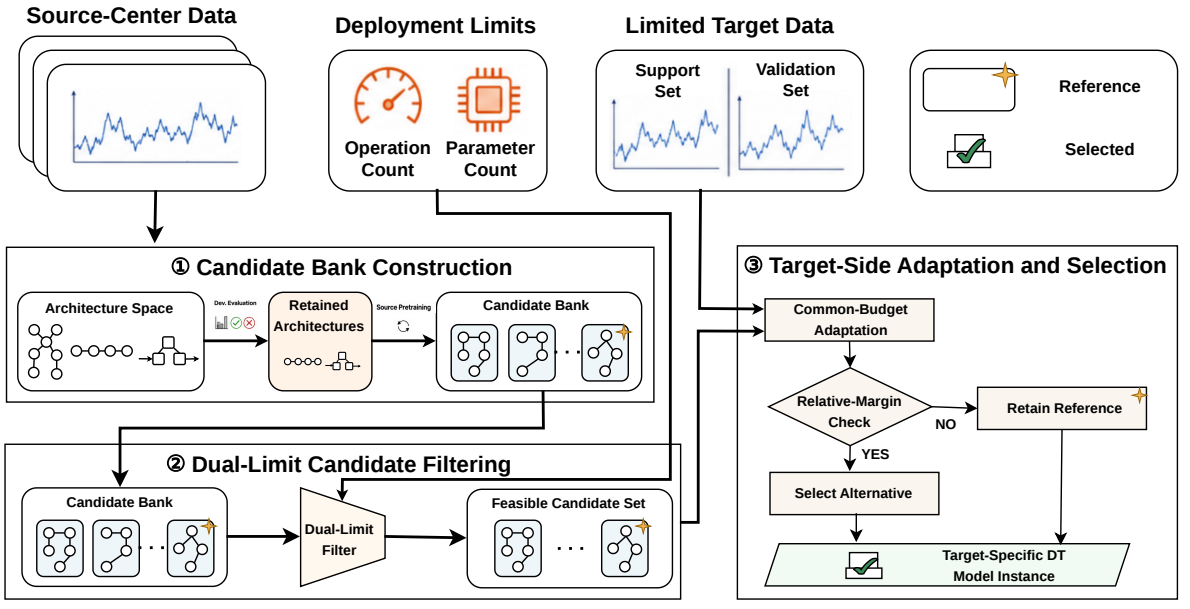


Figure 2: Overall three-stage workflow of MSA-DTI.

4.2. Offline Candidate Bank Construction

MSA-DTI screens the architecture space on architecture-screening cases that are separate from margin calibration and final target evaluation. Each screening case contains support, validation, and check sets, and the validation and check losses are used by the preset screening rule.

A reference architecture $a^{\mathrm{ref}} \in \mathcal{A}$ is fixed before screening and always retained. Alternative architectures are retained according to the preset screening rule. Each retained architecture is then trained on $\mathcal{D}_{\mathrm{src}}$, and each architecture–initialization pair forms a candidate $q = (a_q, \theta_q^0)$ in $\mathcal{Q}$. A retained architecture may therefore contribute more than one candidate when multiple frozen source initializations are kept. One candidate with architecture a^{ref} is fixed as the reference candidate q^{ref}. In Fig. 3, $a^{(1)}, a^{(2)}, \ldots$ denote example architectures in $\mathcal{A}$, $a_1, \ldots, a_M$ denote retained architectures, and $q_1, q_2, \ldots$ denote candidates in $\mathcal{Q}$, with q^{ref} denoting the fixed reference candidate.

The bank and reference candidate are fixed before relative-margin calibration and final target evaluation. Figure 3 summarizes this process. The exact screening and source-training settings are reported in Section 5.1 and Supplementary Section S2.

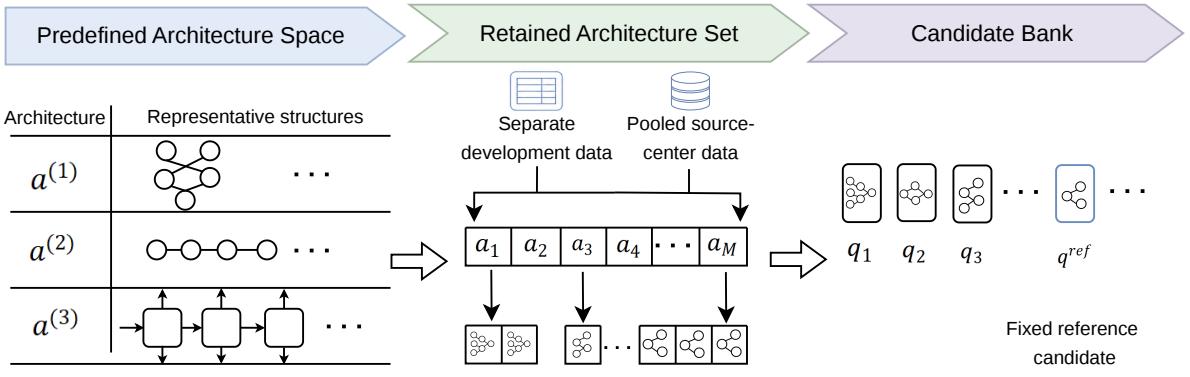


Figure 3: Offline construction of the candidate bank and reference candidate.

4.3. Deployment Filtering

For target center c, MSA-DTI forms $\mathcal{Q}_c^{\mathrm{fea}}$ using Eq. (2) before target adaptation. Operation count and parameter count do not change during parameter adaptation, so candidates outside this set can be removed before target updates. If $q^{\mathrm{ref}} \notin \mathcal{Q}_c^{\mathrm{fea}}$, MSA-DTI returns UNSUPPORTEDLIMIT.

4.4. Target Adaptation

Each feasible candidate starts from θ_q^0. Let t denote the update index, and let $\eta_t > 0$ denote the step size at update t. Candidate q is adapted by

$$\begin{aligned} \theta_{c,q}^{(0)} &= \theta_q^0, \\ \theta_{c,q}^{(t+1)} &= \theta_{c,q}^{(t)} - \eta_t \nabla_\theta \mathcal{L}\left(\mathcal{D}_c^{\mathrm{sup}}; a_q, \theta_{c,q}^{(t)}\right), \\ & \qquad t = 0, \ldots, T-1. \end{aligned} \tag{4}$$

Here, ∇_θ denotes the parameter gradient. The architecture remains fixed during adaptation. All feasible candidates use the same support set and update rule, and their validation losses are then computed using Eq. (3). In the experiments, these gradient updates are implemented using stochastic gradient descent (SGD), as specified in Section 5.1.

4.5. Reference-Based Selection

Because the validation set is small, MSA-DTI uses a preset relative margin so that a marginal post-adaptation loss reduction does not by itself trigger replacement of the fixed reference.

Let $\mathcal{Q}_c^{\mathrm{alt}} = \mathcal{Q}_c^{\mathrm{fea}} \setminus \{q^{\mathrm{ref}}\}$ denote the feasible alternatives. Because this set is finite, MSA-DTI exhaustively evaluates all adapted feasible alternatives. If this set is empty, MSA-DTI returns the adapted reference candidate. Otherwise, let $\widetilde{q}_c \in \arg\min_{q \in \mathcal{Q}_c^{\mathrm{alt}}} J_c(q)$ denote an alternative with the lowest validation loss.

Let $\tau \in [0, 1)$ denote a preset relative margin. The final candidate is

$$q_c^* = \begin{cases} \widetilde{q}_c, & J_c(\widetilde{q}_c) \le (1-\tau) J_c(q^{\mathrm{ref}}), \\ q^{\mathrm{ref}}, & \text{otherwise.} \end{cases} \tag{5}$$

A larger τ requires a larger validation-loss reduction for an alternative to be selected over the reference. The margin affects only model selection and does not change the adapted parameters.

The selected candidate provides the architecture and parameters in Eq. (1). Figure 4 illustrates the adaptation and selection stages.

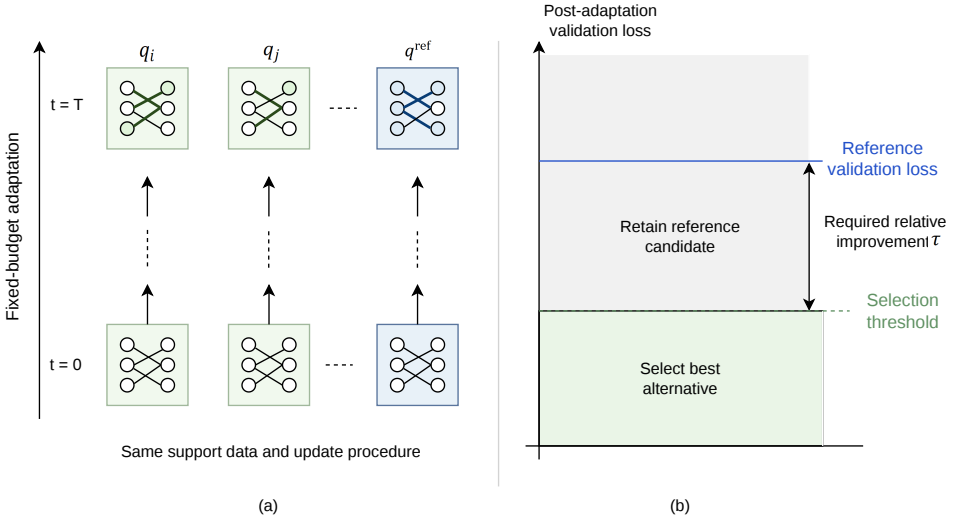


Figure 4: Target adaptation and reference-based model selection.

Algorithm 1 summarizes the target-side procedure.

Algorithm 1 Target-Side MSA-DTI for One Center

Require: Candidate bank $\mathcal{Q}$, reference candidate q^{ref}, support set $\mathcal{D}_c^{\mathrm{sup}}$, validation set $\mathcal{D}_c^{\mathrm{val}}$, deployment limits B_c^F and B_c^P, update number T, step sizes $\{\eta_t\}_{t=0}^{T-1}$, and relative margin τ

Ensure: DT_c or UNSUPPORTEDLIMIT

Compute $\mathcal{Q}_c^{\mathrm{fea}}$ using Eq. (2)
if $q^{\mathrm{ref}} \notin \mathcal{Q}_c^{\mathrm{fea}}$ **then**
    return UNSUPPORTEDLIMIT
end if
for all $q \in \mathcal{Q}_c^{\mathrm{fea}}$ **do**
    Adapt q using Eq. (4)
    Compute $J_c(q)$ using Eq. (3)
end for
$\mathcal{Q}_c^{\mathrm{alt}} \leftarrow \mathcal{Q}_c^{\mathrm{fea}} \setminus \{q^{\mathrm{ref}}\}$
if $\mathcal{Q}_c^{\mathrm{alt}} = \emptyset$ **then**
    $q_c^* \leftarrow q^{\mathrm{ref}}$
else
    Choose $\widetilde{q}_c \in \arg\min_{q \in \mathcal{Q}_c^{\mathrm{alt}}} J_c(q)$
    Select q_c^* using Eq. (5)
end if
$\mathrm{DT}_c \leftarrow \left(a_{q_c^*}, \theta_{c,q_c^*}^{(T)}\right)$
return DT_c

4.6. Relative-Margin Calibration

The margin τ is calibrated on a margin-calibration pool that is disjoint from the architecture-screening cases and final target evaluation. Each calibration case contains support, validation, and check sets. A selection is harmful when the selected alternative has a higher check-set loss than the adapted reference.

MSA-DTI evaluates preset margins and chooses the smallest value that meets the requirements for harmful rate, average check-loss gain, and statistical confidence. The bank, reference candidate, and selected margin are then fixed before final target evaluation. The exact margin grid and calibration criteria are reported in Section 5.1 and Supplementary Section S2.

The 5% harmful-rate requirement is an ex ante design tolerance chosen to limit erroneous alternative replacement during calibration. It is applied to the observed point estimate rather than as a distribution-free safety guarantee. Eligibility also requires both mean check-loss gains to reach 3% and both lower 95% confidence bounds to remain above zero. If no margin is eligible in a new domain, calibrated deployment returns the adapted reference and reports that domain-specific margin calibration is unsupported. A previously frozen margin may still be evaluated as a transfer audit, but that audit is not reported as calibrated reliability for the new domain.

4.7. Analytical Properties

Deployment feasibility. For every supported target limit setting, MSA-DTI returns a candidate only from $\mathcal{Q}_c^{\mathrm{fea}}$,

$$F_c(a_{q_c^*}) \le B_c^F, \qquad P_c(a_{q_c^*}) \le B_c^P.$$

Thus, every returned DT model satisfies both deployment limits.

Required validation improvement. When an alternative is selected and $J_c(q^{\mathrm{ref}}) > 0$, Eq. (5) gives

$$\frac{J_c(q^{\mathrm{ref}}) - J_c(q_c^*)}{J_c(q^{\mathrm{ref}})} \ge \tau.$$

Thus, every selected alternative reaches the preset relative improvement.

A conditional expected-loss result and the target-stage computational complexity are provided in Supplementary Section S3.

5. Experimental Evaluation

We evaluate prediction accuracy, selection reliability, deployment feasibility, target-side cost, and robustness across target conditions. Additional component and transfer analyses are also reported.

5.1. Experimental Setup

5.1.1. Benchmark and Data Protocol

We use a controlled synthetic benchmark for heterogeneous multi-center workloads. Centers differ in workload dynamics, sampling intervals, monitored-variable availability, and observation conditions. Deployment-limit tiers are treated as separate platform-side settings. Types A–C are evaluation strata generated from different parameter ranges for these center-side factors. The detailed generation procedure is given in Supplementary Section S8.

The main evaluation contains 20 source centers and 20 held-out target centers. Two prediction horizons and two support sizes give four cases per target center and 80 held-out cases in total.

Each case is split chronologically into support, validation, check, and test sets, with gaps between adjacent splits. The source, architecture-screening, margin-calibration, diagnostic, and held-out target pools use disjoint center identities. Test data are used only after the architecture and adapted parameters have been fixed. Table 1 reports the main settings.

5.1.2. Candidate Bank, Reference, and Calibration

The initial space contains 66 MLP, TCN, and GRU configurations. A57, a one-layer GRU with 32 hidden units, is fixed as the reference before screening and is also used by PT+FT. An alternative is retained when it outperforms A57 on validation or check data in at least two screening cases. This gives six retained configurations, five executable architectures, and seven candidates.

Table 1 reports the training and target-adaptation settings, and Supplementary Section S2 provides the candidate-bank construction and calibration details.

Table 1: Main experimental configuration.

Category	Item	Setting
Data	Centers; observations; input length	Source / held-out: 20/20; 3000 per center; $L = 96$
	Target cases and data	80 cases; $H \in \{1, 4\}$; $K \in \{10, 20\}$; validation / check / test: 80/60/200
Bank	Initial / retained / executable / candidates	66/6/5/7
	Reference and screening	A57: one-layer GRU-32, fixed before screening and shared with PT+FT; alternative kept after wins in at least two screening cases
	Source-model records	Reference pooled-source initialization; five common alternative initializations; one additional A57 paired control (Supplementary S2)
Training	Source models	Adam / MSE / 50 epochs; learning rate 10^{-3}; batch size 64
	Target adaptation	SGD / MSE / 50 updates; learning rate 0.01; gradient limit 1.0; complete support set
Selection	Grid and eligibility	$\{0.05, 0.075, 0.10, 0.125, 0.15, 0.20\}$; harmful rate $\leq 5\%$; both mean gains $\geq 3\%$; both lower 95% bounds > 0
	Frozen margin	$\tau = 0.10$
Limits	Tight / medium / loose	Operations: $1.5\times10^6/5\times10^6/2\times10^7$; parameters: $3\times10^4/10^5/5\times10^5$
Statistics	Confidence intervals	Center-cluster bootstrap; 4000 repetitions

5.1.3. Task, Baselines, and Metrics

We use workload prediction as the evaluation task because it supports proactive resource management in computing systems [17–19]. Mean squared error (MSE) is the task loss. We also report mean absolute error (MAE), Worst-10% error, conditional value at risk at the 90th percentile (CVaR90), target-side time, and limit satisfaction. Worst-10% is the mean window-level MSE over the 10% of test windows with the largest errors. CVaR90 is the mean case-level MSE over the 10% of target cases with the largest MSE.

For held-out reporting, a selected alternative is labeled harmful when its test MSE is higher than that of the adapted reference. This label is used only after selection. Relative gains are computed for each paired case and then averaged. Confidence intervals use a center-cluster bootstrap with 4000 repetitions, keeping the four cases from one center together.

We compare MSA-DTI with PT+FT, MeDeT-based adaptation, random initialization, few-shot NAS, zero-shot NAS, zero-shot NAS with fine-tuning, and a task-matched H-Meta-NAS-based adaptation [30]. PT+FT is the main controlled baseline because it uses the same source-trained reference and target adaptation procedure as MSA-DTI. H-Meta-NAS-based adaptation is included as the closest architecture-adaptation baseline because it combines few-shot meta-learning with hardware-aware architecture adaptation. We implement it using architecture-indexed first-order meta-initializations learned from the source centers and validation-only target architecture search under the same hard deployment limits. Its feasible search space uses the same 66 architecture definitions, its meta-initializations and search settings are fixed without held-out test data, and the final architecture is chosen using target validation data. The 50-update SGD/MSE budget-matched condition is used in the main comparison; target-side search and adaptation time are included in the reported protocol.

All reported baselines use the same target cases, chronological splits, deployment limits, and architecture definitions. Target-side baseline settings are summarized in Supplementary Table S1, with complete configurations provided in the released implementation.

5.1.4. Runtime Protocol

All methods are implemented in Python and PyTorch and run on an NVIDIA RTX 3060 Laptop GPU. Target-side time includes model creation, initialization loading, target adaptation, validation evaluation, and final selection, but excludes offline preparation and test evaluation. Supplementary Section S5 reports the complete timing and hardware-profiling protocol.

5.2. Overall Performance and Selection Reliability

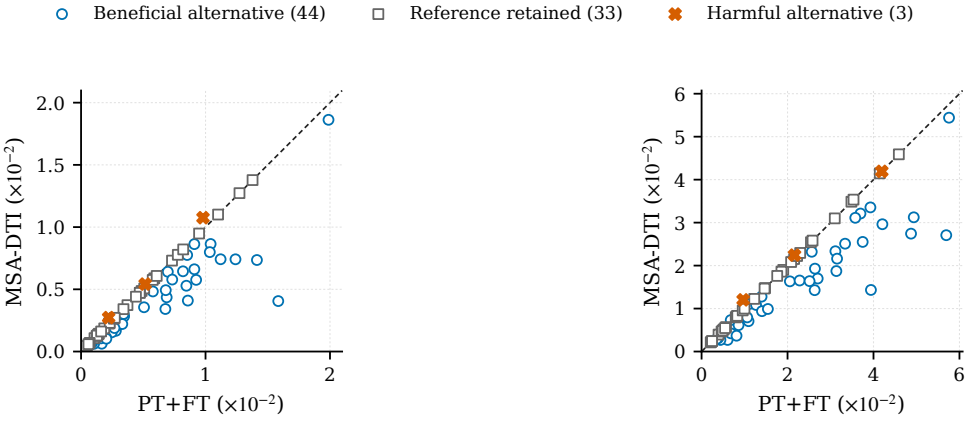
Figure 5 compares prediction performance on the held-out cases. All reported outputs satisfy both deployment limits.

Figure 5: Predictive performance on the held-out target cases: (a) MAE, (b) MSE, (c) Worst-10% error, and (d) CVaR90 (lower is better).

MSA-DTI gives the lowest values for all four error measures. PT+FT is the strongest baseline. Under the 50-update budget-matched setting, the task-matched H-Meta-NAS-based adaptation obtains an MSE of 0.013747, compared with

0.005120 for PT+FT and 0.004219 for MSA-DTI. Relative to PT+FT, MSA-DTI reduces MAE by 8.55%, MSE by 14.60%, and Worst-10% error by 13.76% on average across paired cases. The center-cluster bootstrap 95% confidence interval for the paired MSE reduction is [9.41%, 20.14%].

Figure 6 compares MSA-DTI with the adapted reference at the case level. MSA-DTI retains the reference in 33 cases and selects an alternative in 47 cases. Of these alternatives, 44 have lower test MSE and three have higher test MSE. Harmful selections therefore account for 3.75% of all held-out cases. These test labels do not affect selection. The center-cluster bootstrap 95% confidence interval for the all-case harmful-selection rate is [0.00%, 7.50%]; the interval therefore does not establish a population harmful rate below 5%.



(a) Test MSE

(b) Test Worst-10% error

Figure 6: Held-out paired comparison of MSA-DTI and PT+FT for (a) test MSE and (b) test Worst-10% error; points below the diagonal favor MSA-DTI.

Because the native NAS baselines use different target optimizers and losses, we also run an optimizer-matched diagnostic comparison. All methods included in this diagnostic use SGD, MSE, and 50 target updates. MSA-DTI keeps the lowest MSE, Worst-10% error, and CVaR90. Table 2 moves the central comparison into the main paper, and Supplementary Table S2 gives the complete diagnostic.

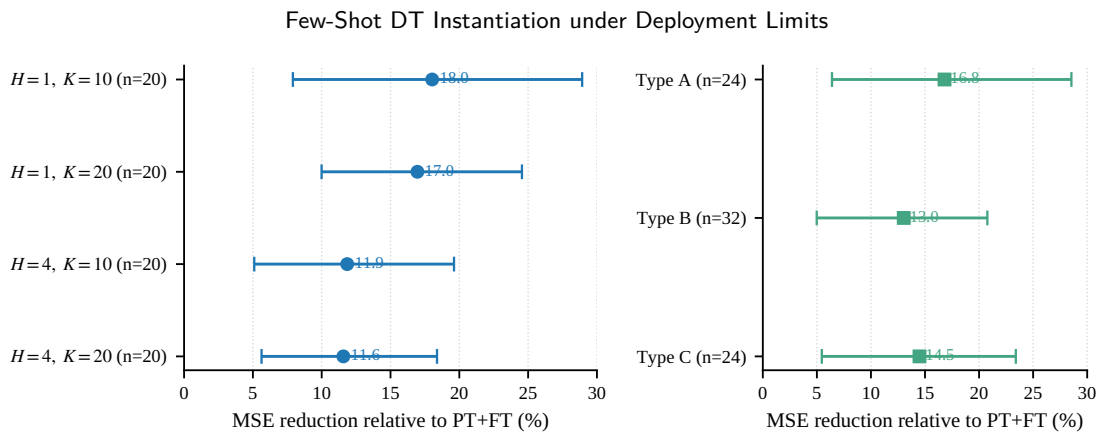
Table 2: Optimizer-matched diagnostic comparison.

Method	MSE↓	Worst-10%↓	CVaR90↓	Adapted models
PT+FT	0.004497	0.017144	0.011878	1.00
Few-shot NAS	0.006715	0.025246	0.019653	12.00
Zero-shot NAS + fine-tuning	0.007402	0.028099	0.020402	12.00
MSA-DTI	**0.003809**	**0.015094**	**0.008696**	6.10

All methods use the same independent diagnostic cases, hard deployment limits, SGD optimizer, MSE loss, and 50 target updates per adapted model. The source initializations and candidate-selection rules remain method-specific. The complete six-method diagnostic is reported in Supplementary Table S2.

5.3. Robustness across Target Conditions

Figure 7 reports paired MSE reductions relative to PT+FT. Positive values favor MSA-DTI.



(a) Horizon and support size

(b) Target center type

Figure 7: Paired MSE reduction relative to PT+FT across (a) horizon/support settings and (b) target-center types; error bars show 95% confidence intervals.

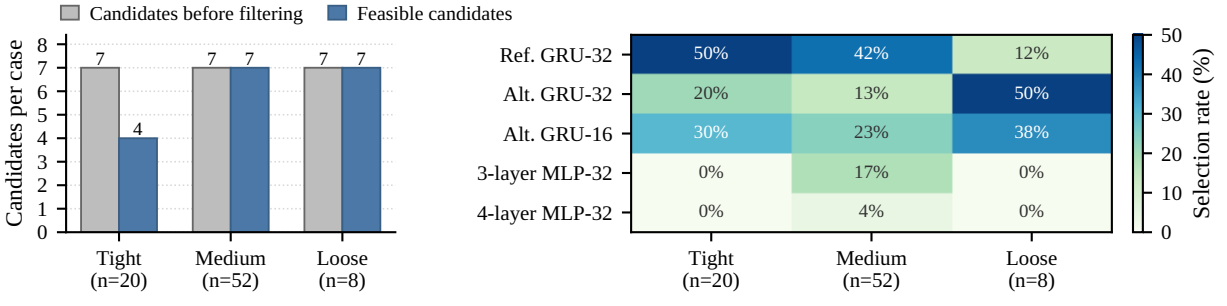
The reduction ranges from 11.57% to 18.03% across the four horizon and support-size settings and from 13.03% to 16.80% across the three center types. All seven confidence intervals remain above zero.

On a separate robustness pool, retraining the fixed candidate architectures with source-training seeds 2904, 2905, and 2906 gives paired MSE reductions of 10.52%, 13.21%, and 11.25%, respectively. Their 95% confidence intervals are [3.93%, 18.17%], [7.62%, 18.89%], and [5.23%, 17.96%]. This analysis tests source-initialization sensitivity after the architecture list has been fixed.

We additionally repeat the complete 66-configuration screening rule on two new 20-center pools, giving three screening samples when the original pool is included, and evaluate each resulting list on a fourth, untouched 20-center pool. The retained-list sizes are 6, 0, and 4; the two new lists have Jaccard similarities of 0 and 0.429 to the frozen six-configuration list. On the independent pool, validation-only diagnostic selection from the frozen list gives a 0.30% mean paired check-MSE reduction with a center-cluster 95% interval of [−0.65%, 1.56%] and a 2.50% harmful-selection rate. Thus, the preset candidate-discovery rule is sample-sensitive and is not claimed to recover a unique architecture set. The frozen main evaluation remains an evaluation of a prespecified bank, while the retained A57 reference and fallback protect deployment when a rescreened pool retains no alternative. Full rescreening results and selection frequencies are reported in Supplementary Section S7.

5.4. Deployment Feasibility and Cost

Figure 8 shows how the three deployment-limit settings affect candidate feasibility and final model selection.



(a) Candidate filtering

(b) Final model selection

Figure 8: Effects of the three deployment-limit settings on (a) the number of feasible candidates and (b) the selected model configurations.

The tight setting reduces the bank from seven candidates to four, whereas the medium and loose settings retain all seven. A single-limit audit in Supplementary Section S5 shows that this reduction is caused by the parameter-count limit: all seven candidates satisfy the operation-count limit, while the tight parameter-count limit excludes three MLP candidates. To examine whether the two limits can independently constrain candidate feasibility beyond the frozen

main bank, Supplementary Section S5 also reports an expanded-bank stress analysis. For both $H = 1$ and $H = 4$, the operation-count and parameter-count limits exclude different candidates: each single-limit feasible set retains seven of eight candidates, whereas their joint feasible set retains six. A corrected target-side stress evaluation uses the same 80 held-out cases, and all outputs selected under the joint limits satisfy both limits. Mean paired MSE reductions relative to PT+FT remain positive under tight, medium, and loose limits: 11.35%, 15.30%, and 18.22%, respectively.

Table 3 reports raw target-side cost and selected-model complexity.

Table 3: Target-side cost and selected-model complexity.

Method	Time per case (s)	Adapted models	Parameters	Estimated operations
PT+FT	0.204 ± 0.004	1.00	5,746	1,050,784
Few-shot NAS	13.471 ± 0.173	12.00	30,995	1,468,894
Zero-shot NAS + fine-tuning	17.601 ± 0.150	12.00	62,076	1,571,621
MSA-DTI	5.676 ± 0.059	6.25	14,892	751,433

Time is mean $\pm$ standard deviation over five runs. Complexity values are averaged over the final models selected in the held-out cases.

MSA-DTI requires 5.676 ± 0.059 seconds per case and adapts 6.25 models on average. It is slower than PT+FT, but is 2.37× faster than few-shot NAS and 3.10× faster than zero-shot NAS with fine-tuning.

Hardware profiling in Supplementary Section S5 reports every executable architecture on an Intel Core i7-12700H CPU and an NVIDIA RTX 3060 Laptop GPU. Median batch-one latency ranges from 0.093 to 1.980 ms on the CPU and from 0.133 to 4.170 ms on the GPU. Serialized model size ranges from 10.3 to 315.9 KiB, and measured peak GPU allocation ranges from 54 to 1037 KiB. Operation count is positively associated with median latency, whereas parameter count exactly orders serialized model size in the measured settings. Peak GPU allocation is not monotonic in parameter count. These measurements support the use of the two counts as distinct complexity indicators, but they do not convert either limit into a hardware latency or memory guarantee or establish cross-platform deployment performance.

5.5. Component Analysis

Removing deployment filtering reduces limit satisfaction from 100% to 96.25%, confirming that filtering is required to enforce the configured hard limits. Direct lowest-validation-loss selection increases the harmful-selection rate from 15.00% to 17.50% on the diagnostic pool, although the confidence intervals overlap. The latter result is therefore an observed difference rather than a statistically confirmed reduction in harmful selection. The reference-only and two-reference-initialization controls in Supplementary Section S4 do not reproduce the cross-architecture gain. Conversely, removing the relative margin leaves the diagnostic MSE nearly unchanged while selecting alternatives more often. The main performance increment is therefore associated with adapting and selecting across the compact architecture bank; the margin is a conservative replacement rule, not the sole source of the accuracy gain. Full component and selection results are reported in Supplementary Section S4.

5.6. External Transfer Evaluation

We also evaluate the frozen method on Alibaba Cluster Trace v2018 [29]. The workload observations come from the production trace, while the deployment-limit tiers remain controlled model-complexity settings. This experiment therefore evaluates external workload-domain transfer rather than a fully measured hardware-deployment setting.

Without recalibrating the margin, the fixed $\tau = 0.10$ rule gives a 4.20% mean paired MSE reduction with a 95% confidence interval of [2.03%, 6.70%]. The harmful-selection rate is 6.25%, which exceeds the original 5% criterion. The result therefore shows a positive average transfer gain but does not reproduce the original reliability criterion. Under the calibrated deployment policy defined in Section 4, this outcome retains the adapted reference rather than treating the frozen margin as certified for Alibaba; the reported frozen-margin result is a transfer audit. The full transfer protocol and results are given in Supplementary Section S7.

6. Conclusion

This paper presented MSA-DTI for few-shot instantiation of the predictive-model component used by a center-specific DT, subject to two per-model deployment limits. MSA-DTI filters a source-trained candidate bank, adapts feasible candidates with a common target budget, and applies reference-based selection. MSA-DTI reduces MSE by 14.60% on average across paired cases relative to PT+FT while satisfying both limits, and 44 of 47 alternative selections

improve test MSE. Independent rescreening shows that candidate discovery is sample-sensitive, so the frozen bank is treated as a prespecified evaluated artifact rather than a uniquely recoverable architecture set; the reference fallback remains essential. The transferred rule also gives a positive mean gain on Alibaba workloads, although its harmful-selection rate exceeds the original criterion and the calibrated deployment policy therefore retains the reference. Future work will develop more stable candidate-discovery procedures, uncertainty-aware selection, and measured deployment limits on additional platforms.

CRediT Authorship Contribution Statement

The final author contribution statement will be added after the author list is confirmed.

Funding

Funding information will be added after confirmation.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data Availability

The source code, frozen configurations, processed results, plot-ready data, and reproduction scripts underlying the reported results are available in GitHub Release v1.2.2: https://github.com/w924871681/paper-code-DT-Twin/releases/tag/v1.2.2. The original Alibaba Cluster Trace v2018 is not redistributed and remains available from its official repository: https://github.com/alibaba/clusterdata. The released documentation specifies preprocessing, disjoint data splits, margin handling, and held-out evaluation.

Declaration of Generative AI and AI-Assisted Technologies in the Manuscript Preparation Process

During the preparation of this work, the authors used OpenAI ChatGPT to assist with language refinement, manuscript organization, and readability improvement. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

References

[1] L. U. Khan, Z. Han, W. Saad, E. Hossain, M. Guizani, and C. S. Hong, "Digital twin of wireless systems: Overview, taxonomy, challenges, and opportunities," *IEEE Communications Surveys & Tutorials*, vol. 24, no. 4, pp. 2230–2254, Fourth Quarter 2022, doi: 10.1109/COMST.2022.3198273.

[2] S. Mihai, M. Yaqoob, D. V. Hung, W. Davis, P. Towakel, M. Raza, M. Karamanoglu, B. Barn, D. Shetve, R. V. Prasad, H. Venkataraman, R. Trestian, and H. X. Nguyen, "Digital twins: A survey on enabling technologies, challenges, trends and future prospects," *IEEE Communications Surveys & Tutorials*, vol. 24, no. 4, pp. 2255–2291, Fourth Quarter 2022, doi: 10.1109/COMST.2022.3208773.

[3] H. Xu, J. Wu, Q. Pan, X. Guan, and M. Guizani, "A survey on digital twin for Industrial Internet of Things: Applications, technologies and tools," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 4, pp. 2569–2598, Fourth Quarter 2023, doi: 10.1109/COMST.2023.3297395.

[4] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, "Digital twin in industry: State-of-the-art," *IEEE Transactions on Industrial Informatics*, vol. 15, no. 4, pp. 2405–2415, 2019, doi: 10.1109/TII.2018.2873186.

[5] B. Qin, H. Pan, Y. Dai, X. Si, X. Huang, C. Yuen, and Y. Zhang, "Machine and deep learning for digital twin networks: A survey," *IEEE Internet of Things Journal*, vol. 11, no. 21, pp. 34694–34716, Nov. 2024, doi: 10.1109/JIOT.2024.3416733.

[6] A. Hakiri, A. Gokhale, S. Ben Yahia, and N. Mellouli, "A comprehensive survey on digital twin for future networks and emerging Internet of Things industry," *Computer Networks*, vol. 244, Art. no. 110350, 2024, doi: 10.1016/j.comnet.2024.110350.

[7] Y. Wu, K. Zhang, and Y. Zhang, "Digital twin networks: A survey," *IEEE Internet of Things Journal*, vol. 8, no. 18, pp. 13789–13804, Sept. 2021, doi: 10.1109/JIOT.2021.3079510.

[8] J. Li, S. Guo, W. Liang, J. Wang, Q. Chen, Y. Zeng, B. Ye, and X. Jia, "Digital twin-enabled service provisioning in edge computing via continual learning," *IEEE Transactions on Mobile Computing*, vol. 23, no. 6, pp. 7335–7350, 2024, doi: 10.1109/TMC.2023.3332668.

[9] A. Vettoruzzo, M.-R. Bouguelia, J. Vanschoren, T. Rögnvaldsson, and K. C. Santosh, "Advances and challenges in meta-learning: A technical review," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 7, pp. 4763–4779, 2024, doi: 10.1109/TPAMI.2024.3357847.

[10] H. Gharoun, F. Momenifar, F. Chen, and A. H. Gandomi, "Meta-learning approaches for few-shot learning: A survey of recent advances," *ACM Computing Surveys*, vol. 56, no. 12, Art. no. 294, pp. 1–41, 2024, doi: 10.1145/3659943.

[11] H. Sartaj, S. Ali, and J. M. Gjøby, "MeDeT: Medical device digital twins creation with few-shot meta-learning," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 6, Art. no. 158, pp. 1–36, 2025, doi: 10.1145/3708534.

[12] M. Tan, B. Chen, R. Pang, V. Vasudevan, M. Sandler, A. Howard, and Q. V. Le, "MnasNet: Platform-aware neural architecture search for mobile," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 2820–2828, doi: 10.1109/CVPR.2019.00293.

[13] B. Wu, X. Dai, P. Zhang, Y. Wang, F. Sun, Y. Wu, Y. Tian, P. Vajda, Y. Jia, and K. Keutzer, "FBNet: Hardware-aware efficient ConvNet design via differentiable neural architecture search," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 10734–10742, doi: 10.1109/CVPR.2019.01099.

[14] Y. Zhao, L. Wang, Y. Tian, R. Fonseca, and T. Guo, "Few-shot neural architecture search," in *Proc. 38th Int. Conf. Mach. Learn. (ICML)*, vol. 139 of *Proc. Mach. Learn. Res.*, pp. 12707–12718, 2021.

[15] G. Li, D. Hoang, K. Bhardwaj, M. Lin, Z. Wang, and R. Marculescu, "Zero-shot neural architecture search: Challenges, solutions, and opportunities," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 12, pp. 7618–7635, Dec. 2024, doi: 10.1109/TPAMI.2024.3395423.

[16] M.-T. Wu, H.-I. Lin, and C.-W. Tsai, "A training-free neural architecture search algorithm based on search economics," *IEEE Transactions on Evolutionary Computation*, vol. 28, no. 2, pp. 445–459, Apr. 2024, doi: 10.1109/TEVC.2023.3264533.

[17] Y.-M. Kim, S. Song, B.-M. Koo, J. Son, Y. Lee, and J.-G. Baek, "Enhancing long-term cloud workload forecasting framework: Anomaly handling and ensemble learning in multivariate time series," *IEEE Transactions on Cloud Computing*, vol. 12, no. 2, pp. 789–799, Apr.–Jun. 2024, doi: 10.1109/TCC.2024.3400859.

[18] F. Zhao, W. Lin, S. Lin, S. Tang, and K. Li, "MSCNet: Multi-scale network with convolutions for long-term cloud workload prediction," *IEEE Transactions on Services Computing*, vol. 18, no. 2, pp. 969–982, Mar.–Apr. 2025, doi: 10.1109/TSC.2025.3536313.

[19] B. Feng and Z. Ding, "Application-oriented cloud workload prediction: A survey and new perspectives," *Tsinghua Science and Technology*, vol. 30, no. 1, pp. 34–54, Feb. 2025, doi: 10.26599/TST.2024.9010024.

[20] L. M. Zintgraf, K. Shiarlis, V. Kurin, K. Hofmann, and S. Whiteson, "Fast context adaptation via meta-learning," in *Proc. 36th Int. Conf. Mach. Learn. (ICML)*, vol. 97 of *Proc. Mach. Learn. Res.*, pp. 7693–7702, 2019.

[21] Y. Zhao, T. Zhang, J. Li, and Y. Tian, "Dual adaptive representation alignment for cross-domain few-shot learning," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 10, pp. 11720–11732, 2023, doi: 10.1109/TPAMI.2023.3272697.

[22] J. Wu, X. Liu, X. Yin, T. Zhang, and Y. Zhang, "Task-adaptive prompted transformer for cross-domain few-shot learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 6, pp. 6012–6020, 2024, doi: 10.1609/aaai.v38i6.28416.

[23] W. Wang, L. Duan, Y. Wang, J. Fan, and Z. Zhang, "MMT: Cross domain few-shot learning via meta-memory transfer," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 12, pp. 15018–15035, Dec. 2023, doi: 10.1109/TPAMI.2023.3306352.

[24] J. Son, S. Lee, and G. Kim, "When meta-learning meets online and continual learning: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 47, no. 1, pp. 413–432, 2025, doi: 10.1109/TPAMI.2024.3463709.

[25] Z. Zuo, Y. Huang, Z. Li, Y. Jiang, and C. Liu, "Mixed contrastive transfer learning for few-shot workload prediction in the cloud," *Computing*, vol. 107, no. 1, Art. no. 5, 2025, doi: 10.1007/s00607-024-01366-y.

[26] Y. Zhao, L. Wang, and T. Guo, "Multi-objective neural architecture search by learning search space partitions," *Journal of Machine Learning Research*, vol. 25, no. 177, pp. 1–41, 2024.

[27] M. Emami Khansari and S. Sharifian, "A deep reinforcement learning approach towards distributed Function as a Service (FaaS) based edge application orchestration in cloud-edge continuum," *Journal of Network and Computer Applications*, vol. 233, Art. no. 104042, 2025, doi: 10.1016/j.jnca.2024.104042.

[28] D. Hortelano, I. de Miguel, R. J. Durán Barroso, J. C. Aguado, N. Merayo, L. Ruiz, A. Asensio, X. Masip-Bruin, P. Fernández, R. M. Lorenzo, and E. J. Abril, "A comprehensive survey on reinforcement-learning-based computation offloading techniques in Edge Computing Systems," *Journal of Network and Computer Applications*, vol. 216, Art. no. 103669, 2023, doi: 10.1016/j.jnca.2023.103669.

[29] [dataset] Alibaba Group, "Alibaba Cluster Trace Program: Cluster Trace v2018," GitHub repository, 2018. [Online]. Available: https://github.com/alibaba/clusterdata/tree/master/cluster-trace-v2018. Accessed: Jul. 3, 2026.

[30] Y. Zhao, X. Gao, I. Shumailov, N. Fusi, and R. Mullins, "Rapid model architecture adaption for meta-learning," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, doi: 10.52202/068431-1360.

[31] D. Lian, Y. Zheng, Y. Xu, Y. Lu, L. Lin, P. Zhao, J. Huang, and S. Gao, "Towards fast adaptation of neural architectures with meta learning," in *Proc. International Conference on Learning Representations (ICLR)*, 2020.

[32] P. Eustratiadis, L. Dudziak, D. Li, and T. Hospedales, "Neural fine-tuning search for few-shot learning," in *Proc. International Conference on Learning Representations (ICLR)*, 2024.